Checkpoint 3 Summary

Group 5
Hussain Sulaiman Zia 26100011
Shanze Sajid 28100025
Daniyal Hussain Shah 28100434

Video Link: Signals
(Uploaded the same video twice - 2nd time with reduced quality, because the first time it seemed to not be opening on Drive)

Hardware Setup and Observed Latency

We ran the real-time demo using standard laptop hardware (built-in microphone and speakers). While our algorithm was built for 16 kHz mono audio, the laptop hardware natively captured at 48 kHz stereo. We handled this by forcing downsampling and channel-flattening in the code. We also manually disabled the OS-level audio enhancements (like built-in echo cancellation) to ensure our algorithm was processing the raw acoustic environment (else our output was turning out completely clean).

Latency was our biggest bottleneck. Testing revealed that our hardware delay fluctuated wildly—ranging from just a few milliseconds to over 240ms—due to OS-level audio buffering. This drifting delay made it incredibly difficult to keep the signals synced in real-time.

Analysis: Simulation vs. Real-Time Performance

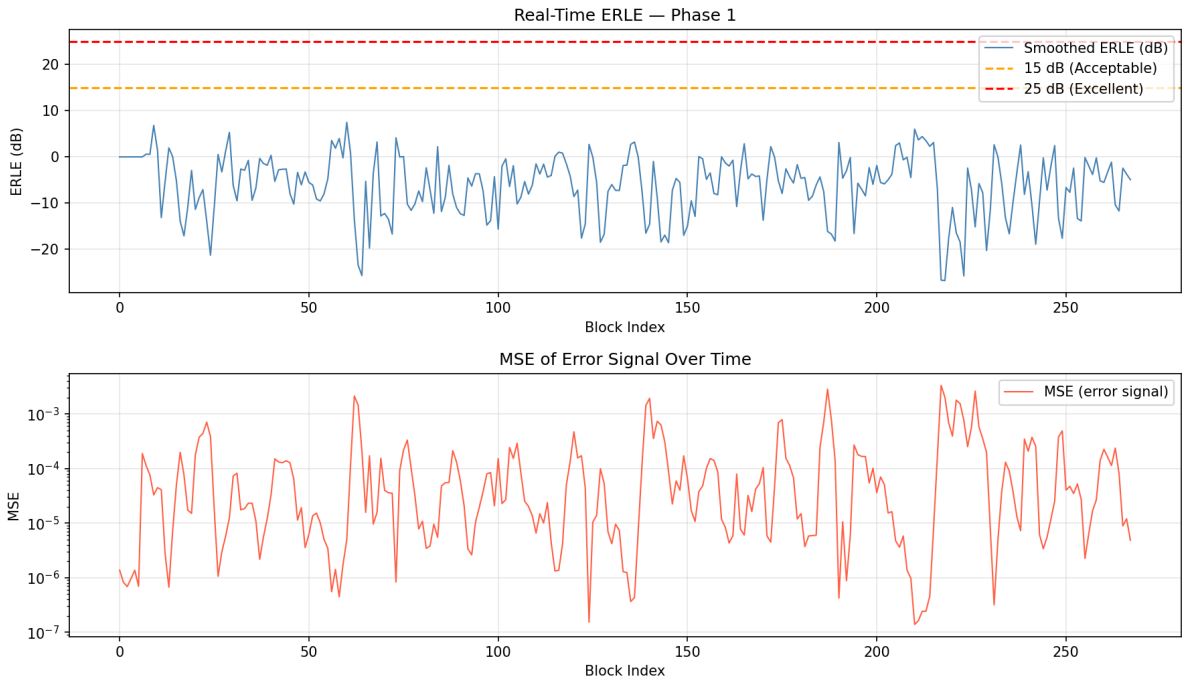
Simulation Results Under ideal simulation conditions, the NLMS algorithm performed extremely well. We achieved a final ERLE of **23.98 dB** (without near-end noise) and **18.50 dB** (with near-end noise). Because the simulation had access to the entire audio file at once, the algorithm benefited from perfect time alignment, zero background noise, and no data interruptions.

Real-Time Challenges During the live demo, the system struggled to maintain convergence and frequently hit negative ERLE values. This gap between theory and reality came down to a few physical constraints:

- **The "Block" Barrier:** The live demo processed audio in small 512-sample chunks. Because the filter couldn't easily look back at the *previous* block's history, it essentially started partially "blind" at the beginning of every new chunk, causing periodic drops in performance.
- **Drifting Alignment:** Because the laptop's audio delay was unpredictable, the algorithm frequently tried to cancel an echo using reference audio that was either

slightly too early or too late. When you subtract unaligned audio, it actually *adds* noise, which mathematically results in negative ERLE.

- We tried multiple methods to measure this delay value, but still were unable to solve the issue



So, despite the fact we used the same NLMS Algo from the previous deliverable, the results of our demo were not good at all.